

UNIVERSITY OF WATERLOO
FACULTY OF ENGINEERING
DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

ECE498A Detailed Design

INDOOR ROBOT NAVIGATION SYSTEM
2015.024

(names removed)

Consultant: Prof. Patrick Mitran

Submitted: July 3, 2014

This is a very strong, although quite dense, Detailed Design report. Throughout you will find excellent examples of justifications and analysis.

Table of Contents

Table of Figures.....	2
Tables of Tables.....	2
1 - High Level Description	3
1.1 - Motivation.....	3
1.2 - Problem Objective	3
1.3 - Block Diagram.....	3
2 - Project Specifications	5
2.1 - Functional Specifications.....	5
2.2 - Non-functional Specifications.....	6
3 - Detailed Design	7
3.1 - Ultrasound Transceiver Interface.....	7
3.2 - Ultrasound & RF Multiple Access Control and Beacon/Receiver Synchronization	9
3.3 - Speed of Sound Detector.....	14
3.4 - Navigation & Path-Finding System	15
3.5 - Robot Control System.....	20
3.6 - Robot Control Interface	23
4 - Discussion on Design	27
4.1 - Discussion on Design.....	27
4.2 - Project Timeline.....	29

Table of Figures

Figure 1: Indoor Robot Navigation System Block Diagram.....	4
Figure 2: Ultrasound Transmitter Interface Designs.....	8
Figure 3: Ultrasound Receiver Interface Designs	8
Figure 4: Transceiver Interface Testing at Distance of 20 cm	9
Figure 5: Robot Circuitry	20
Figure 6: Unity Feedback Motor Control System.....	22
Figure 7: Root Locus Plot of Motor Control System.....	22
Figure 8: Step Response of Motor Controller.....	23
Figure 9: Entity Relationship Diagram for Robot Control Interface	26

Tables of Tables

Table 1: Functional Specifications of Indoor Robot Navigation System	6
Table 2: Non-Functional Specifications of Indoor Robot Navigation System	6
Table 3: Summary of Beacon Synchronization and Multiple Access Control Methods.....	14
Table 4: Summary of Speed of Sound Detection Methods.....	15
Table 5: Evaluation of Map Representation Schemes for Navigation	16
Table 6: Navigation Algorithm Analysis Metrics.....	17
Table 7: Navigation Algorithm Simulator Maps	18
Table 8: Navigation Algorithm Simulation Times – First Pass	18
Table 9: Navigation Algorithm Simulation Times – Second Pass.....	19
Table 10: Analysis of Computation Cost per Cell Move in Robot Navigation	19
Table 11: Decision Matrix for Navigation Algorithm	19
Table 12: Robot Speed Measurements	21
Table 13: Summary of Topology Analysis.....	25
Table 14: Project Timeline	29

1 - High Level Description

1.1 - Motivation

Autonomous robotics are becoming very common in today's society. In fact, yearly sales for household robots worldwide reached about 2 million in 2012 [1]. This number is expected to increase significantly in the near future. However, one of the greatest challenges in developing an autonomous robot is performing autonomous navigation. This is often constrained by limitations in knowledge of the environment such as the robot's current position. Even when traversing a known space, a robot's position relative to that space is critical to travelling efficiently. Without this, the best it could do is approximate its position using heuristics (i.e., collisions) or resort to stochastic motion, which imposes uncertainty when path-finding or maximizing area coverage (i.e., a robot vacuum [2]).

In order to facilitate efficient use of indoor autonomous robots, it is imperative to build a general system that can be used to determine a robot's location indoors. A typical solution, like the Global Positioning System (GPS), has an accuracy of about 3 meters in ideal open-air conditions [3]. This is not practical for a small robot travelling indoors. Furthermore, GPS often operates poorly in enclosed environments due to radio frequency signal attenuation and reflection. Thus, indoor positioning requires a different approach.

A system like this could be applied to an office space. If a robot knows its location and if it can detect obstacles, it could efficiently deliver parcels in an indoor area. In a large building, this would reduce employee time wasted walking across the office. It would also benefit people who are injured that need to transport an item from one place to another. In short, such a robotic delivery system could reduce unproductive travel time in an office.

1.2 - Problem Objective

The objective of the project is to design an indoor positioning and navigation system for use by an autonomous robot. These systems are exemplified through the use of a robot performing point-to-point navigation as it could be applied to intra-office delivery. In short, the objective is to design and apply a robot navigation system.

Autonomous robots require the ability to calculate their location with precision in a coordinate space in order to perform path-planning. Hence, the first project sub-objective is to develop a system in which a robot may determine its current position. It is necessary that there be a reasonable upper bound on position error with respect to the size of the robot and the size of its enclosed operation area. The position update frequency should have a lower bound in order to track the position of a robot in motion. Furthermore, any positioning system should be feasible for deployment in an existing indoor environment.

An autonomous robot itself also requires a means of navigation. This involves using various sensors such as a positioning system in order to perform path-planning, object-avoidance and ultimately machine-learning. Therefore, the second sub-objective is to develop such a system suitable for point-to-point navigation of an unknown indoor map. This system would consume the location data provided by the designed positioning system.

1.3 - Block Diagram

The overall structure of the indoor robot navigation system is described in **Error! Reference source not found.** below. The diagram consists of 4 main sections: Beacon, Receiver, Robot, and User Interface. The Beacon and Receiver are the two sub-systems which implement the indoor positioning system underlying the navigation system. The Beacon section describes a device that transmits an ultrasonic signal

and coordinates transmissions with other beacons. The Receiver section is a device that receives ultrasound signals and then calculates its position relative to the beacons. The components in the block diagram that cross the boundary between Receiver and Beacon exist identically in both sub-systems. In general, the Receiver would be affixed to the robot and a constellation of Beacons would be installed at fixed locations. The Robot section describes a device that will use data from the Receiver and other sensors to perform navigation towards a destination. This destination would be provided using a User Interface software application, which would also display the current position of the Robot relative to the indoor positioning system's coordinate system.

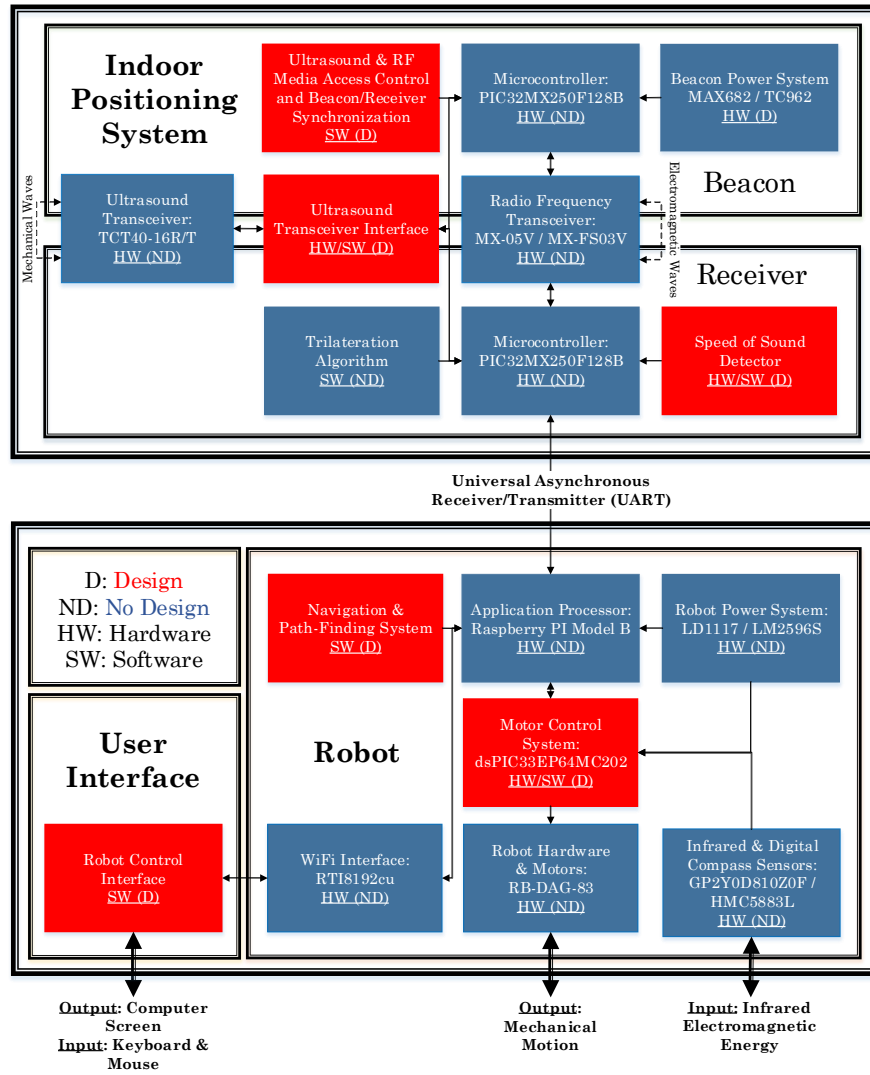


Figure 1: Indoor Robot Navigation System Block Diagram

Some of the components on the block diagram, such as the microcontroller or Radio Frequency (RF) Transceiver, are fairly obvious in their function. However, others are more specific to this project and require explanation. The block labelled as, “Ultrasound & RF Media Access Control and Beacon/Receiver Synchronization” refers to a set of coordination protocols implemented in software that implement the Indoor Positioning System. For example, when measuring the distance between a beacon and the receiver, the beacon would send an ultrasound pulse, and depending on the implementation, a data frame over RF. If another beacon also sends an ultrasonic pulse during this time, the receiver may detect the second pulse

before the first pulse. This would result in an incorrect measurement of distance. Thus, RF and ultrasound media access control and possibly time synchronization is necessary across a distributed system of beacons and receivers. This also impacts the scalability of the system in terms of the number of beacons and receivers, accuracy, and update frequency.

The “Trilateration Algorithm” block refers to the method in which the receiver detects ultrasound pulses, rejecting erroneous signals such as noise, and then performs trilateration using the computed distances to three or more detected beacons. This needs to take into consideration possible errors while an object is moving and background ultrasound noise. To determine the distance to a beacon using a time-of-flight approach, for which the travel time of the ultrasound signal is known, the speed of sound must also be known. Thus, the block “Speed of Sound Detector” performs this function. There are two other blocks in the Beacon and Receiver sections for ultrasound communication. The “Ultrasound Transceiver Interface” block is primarily a hardware interface such that the microcontroller may be able to transmit or receive ultrasonic signals using the Ultrasound Transceiver transducer pair.

Finally, the “Navigation & Path-Finding System” refers to the software used by the robot to calculate a path from one arbitrary point in a room to another. There may be unexpected obstacles in the way, so this algorithm must have a way of avoiding unexpected problems while still finding its way to its destination. In addition, to minimize travelling time, the robot should be actively searching for the shortest path.

2 - Project Specifications

Table 1 in Section 2.1 refers to the functional specifications which define the core requirements and behaviour of the system. Table 2 in Section 2.2 includes the non-functional specification which delineate the non-core requirements. To facilitate implementation of a robot navigation system in the limited amount of time, the scope of the project was constrained. The project design and specification take into consideration scalability in terms of the number of beacons and robots, but the prototype is limited to coverage of a small area. Furthermore, accuracy specifications for ultrasound positioning were adapted from results in published academic works [4] [5]. However, it was necessary to relax these specification due to constrained time in which to iterate on design.

2.1 - Functional Specifications

Most of the functional specifications in Table 1 are easily verifiable. Functional specifications 1 through 8 directly refer to either quantifiable metrics or basic functionality. Basic functionality (1, 2, 5) can be verified by deploying beacons and determining whether all receivers produce correct position data under the test conditions. Correctness refers to meeting the strictly quantitative metrics. Quantitative metrics such as accuracy (6, 7), frequency (8), and detection area (3, 4) can be evaluated empirically by recording debug output from the receiver. For example, position error could be determined by comparing to a known source (i.e., a metre stick). Verifying tracking of moving object poses greater difficulty; however, one approach would be to determine the correlation of the position measurements to a straight-line when the robot is moving straight. It is worth noting that metrics 2 and 5 are non-essential since they are required for the system to be scalable. The system is minimally operational with only three beacons and one receiver.

Functional Specifications 9 through 14 refer to metrics required for the robot and interaction with the robot. Specification 9 refers to the functionality of the robot which is to navigate to a destination specified by the user (per specification 13) carrying up to the mass given in specification 10. The tolerance specified by number 9 is reasonable considering the tolerances of the positioning sub-system and mechanical nature

of the robot. Specification 11 refers to the robot's ability to navigate accurately while the ultrasonic receiver is not receiving an update. This is helpful to the path finding algorithm. Specification 12 is necessary to prevent collisions that may damage the robot and to generate a map of the known area. Finally, specification 14 is associated with 13 given that the user should know where the robot is prior to giving a command, but it is not essential for robot navigation.

Table 1: Functional Specifications of Indoor Robot Navigation System

	Description	Essential?
1	IPS Receiver must provide a position in a coordinate system to its device using three or more beacons.	Essential
2	IPS should allow multiple (e.g., two or more) receivers to detect and provide a position to their device concurrently while operating in the same environment.	Non-Essential
3	A system of three-beacons must provide a coverage area to a receiver at floor level of a radius of at least 1.5 m.	Essential
4	Beacon ultrasound must be detectable at a Receiver from a distance of at least 4 m.	Essential
5	IPS Beacons must operate as a distributed system and scale with the addition or failure of beacons; it must operate with four or more beacons.	Non-Essential
6	IPS Receiver on a stationary robot can determine its position with accuracy better than ± 8 cm.	Essential
7	IPS Receiver can determine position within ± 15 cm when moving up to 10 cm/s.	Essential
8	IPS Receiver can provide position at a frequency of at least 2 Hz	Essential
9	Robot can navigate from a point (within range of beacons) in a flat indoor space to another point while avoiding obstacles. Robot must reach target within ± 15 cm.	Essential
10	Robot should be able to carry 250 g of payload	Non-Essential
11	Robot should be able to move in a straight line to a destination within ± 10 cm accuracy independent of the Indoor Positioning System.	Non-Essential
12	Robot should be able to stop abruptly in the event that an unexpected obstacle is encountered less than 10 cm in front of the robot.	Essential
13	User must be able to give the robot a destination from a graphical user interface.	Essential
14	A user interface must display the location of the robot within the coordinate system.	Non-Essential

2.2 - Non-functional Specifications

Table 2: Non-Functional Specifications of Indoor Robot Navigation System

	Description	Essential?
1	Beacons should be able to operate for at least 24 continuous hours from a battery source.	Non-Essential
2	Robot should be able to operate continuously for one hour from a battery source.	Essential
3	Beacon should use less than 100 mA peak current across all voltage rails.	Essential
4	Size of each receiver is at most 8 cm x 8 cm x 5 cm	Essential
5	Size of each beacon is at most 8 cm x 8 cm x 5 cm	Non-Essential
6	Weight of receiver is at most 150 g	Essential
7	Weight of beacon is at most 200 g	Non-Essential

In Table 2, the size and weight specifications of the beacon are non-essential because the beacon is a stationary device. That is, a mount could be created to support the device in a given situation. Therefore, it is not necessary to set a strict limit on the size and weight of the beacon. In the case of the receiver, these limits are necessary since it is meant to be mounted on a mobile robot. Ideally, it will not interfere with the robot's normal operations. In terms of power, the beacons are meant to have long operation times that may require being powered from batteries in order to accommodate temporary deployment. Thus maximizing beacon power efficiency is important to avoid frequent battery replacements in potentially hard-to-reach devices. However, in contrast to the specification of robot battery-life, the endurance of the beacon is non-essential given that alternatives such as AC power would be feasible. The cost of beacons and receivers

prototypes is non-objective and is therefore not specified. Robot cost is not specified as it is largely dependent on the end-application.

3 - Detailed Design

Section 1.3 provides an overview of the system design employed, while the following sections outline the elaborate on the design of the sub-systems. Sections 3.1 to 3.3 describe design components of the indoor positioning system, while Sections 3.4 to 3.5 describe those of the robot and Section 3.8 describes the system in which robots are coordinated.

3.1 - Ultrasound Transceiver Interface

3.1.1 - Ultrasound Transducer Background

The IPS is designed to function where Beacons are coordinated to transmit ultrasound pulses to be detected by the Receiver. To interface these transducers with the PIC32MX250F128B, analog circuitry is required to amplify or generate the transmitted signal, and to amplify and filter the received signal. This controller is equipped with pulse-width modulation (PWM) outputs at 3.3 V and 10 mA, analog-to-digital converters (ADC), but not a digital-to-analog converter (DAC) [6].

Ultrasound transmitter and receiver transducers typically act as narrow bandwidth band-pass filters around a center frequency. The chosen parts are rated for 40 kHz with a bandwidth of 1.2 kHz. Electrically, the transmitter acts like an impedance of about 900 Ω and receiver produces a small voltage on mechanical stimulation. These transducers are characterized by a conical transmission pattern with a 70° arc around the focal point, attenuating with increasing angles. Sound pressure is inversely proportional to distance and attenuation increases with frequency [7]. Therefore, it is important to transmit at a high amplitude and close to the center frequency to maximize transmission range, and amplify the received signal with sufficient gain. This is necessary to satisfy the minimum 4 m detection range and 1.5 m detection radius functional specifications. The latter would be implicitly met given the 70° arc if $4\text{ m} \leq 1.5/\sin(35^\circ) = 2.6\text{ m}$; that is, a 4 m range should allow a detection radius of 2.3 m at a height 3.3 m above the Receiver. Thus it remains necessary to show that range exceeds 4 m in order to satisfy functional specifications 3 & 4.

3.1.2 - Transmitter Interface

The transmitter interface should have the ability such that the microcontroller can generate an ultrasound pulse sequence using the transducer. Firstly, it is necessary to drive the transducer with an appropriate voltage and sufficient transmission power. The transducer can be modelled as an impedance. Thus, a simple device such as a comparator or transistor should drive the transducer. Secondly, it is important to consider how the signal should be generated. Clearly, a 40 kHz signal must be generated in order operate the transducer near its center frequency to minimize attenuation and power loss. A strategy taken by low-cost ultrasonic range modules (i.e., HC-SR04) is to generate a square wave and amplify it with some form of drive circuitry. The generation could be accomplished using an oscillator and an appropriate frequency divider or a simple device such as a 555 timer [8] configured to pulse at 40 kHz with 50% duty cycle in a stable mode. Figure 2(a) demonstrates such a system using a single BJT to drive the transducer. A problem with this design is that the Beacon would need to schedule timer interrupts to release the enable trigger. A strategy could be taken like the HC-SR04 to emit only 8 pulses. This could be implemented using a binary ripple counter. However, the design would become unnecessarily complex, where a third design in Figure 2(b) is simply a BJT transistor which can be driven by a microcontroller output pin in software (bit-banging) or through built-in hardware pulse width modulation (PWM). A slight variation on (b) could use a MAX3232 [9] RS-232 to TTL adapter to drive the transducer. The advantage of this is that MAX3232 includes a buck-boost voltage regulators to increase the digital PWM signal from 0 V ~ 3.3 V to -5 V ~ 5

V between the outputs. Thus no high voltage rail would be required; however, the MAX3232 is of little use since wireless transmission already requires a 9 V ~ 12 V rail. Ultimately (b) was chosen since it is the simplest and cheapest design of the four as its only semiconductor is a simple BJT. PWM should also be more accurate with respect to frequency than a 555 timer as it would rely on the Microcontroller's crystal oscillator rather than potentially inaccurate resistors and capacitors.

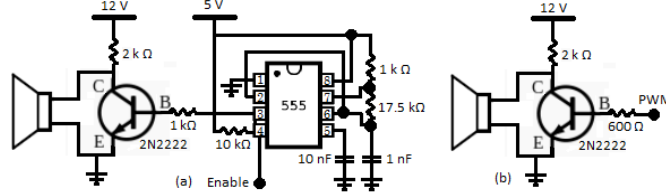


Figure 2: Ultrasound Transmitter Interface Designs

3.1.3 - Receiver Interface

The transmitter interface should have the ability such that the microcontroller can detect pulses received by the transducer. A simple approach would be to construct an amplifier which raises the sensor about to a suitable level relative to the voltage rail of the microcontroller. For example, an amplifier could be implemented using several cascaded BJT amplifier stages followed by a comparator such that the output signal is digital. That is, it would output logical '0' if the signal is below a threshold and logical '1' if the signal is above a threshold. This is necessary since the input signal is essentially using binary antipodal encoding. That is, the difference between the phase of the pulse is differentiated by signal amplitude. Since a comparator is essentially an op-amp, an obvious alternative would be to replace the transistors with one more high gain amplification stage using an op-amp followed by a final amplifier employed as the comparator. A design using two high gain stages, with an effective gain of 5000 is shown in Figure 3(a). A third alternative worth noting is that the comparator could be omitted by connecting the output of the gain stages to an ADC on the microcontroller. This would provide more flexibility in terms of signal processing but it would impose a larger computational cost. Since rail-to-rail op-amps in one package are relatively cheap (i.e., MCP6024) and require fewer gain stages, (a) was chosen for the design.

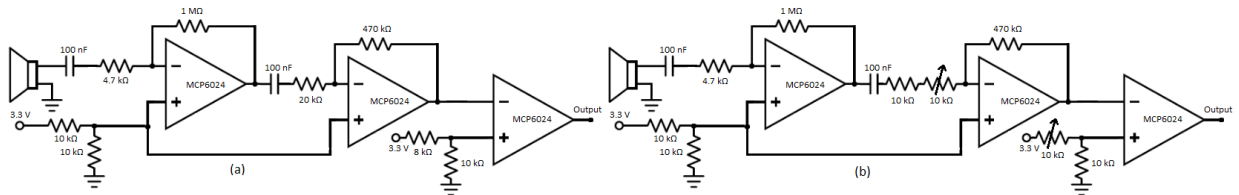


Figure 3: Ultrasound Receiver Interface Designs

3.1.4 - Hardware Testing and Revision

The designs in Figure 2(b) and Figure 3(a) for the transmitter and receiver interfaces were constructed on a breadboard and then tested by placing the transducers at various distances apart and observing the output using an oscilloscope. Functional specification 4 for detection range greater than 4 m was met, and thus 3 for detection radius greater than 1.5 m was met. Figure 4 shows the output of the receiver interface in blue and the input to the transmitter transducer in pink at a distance of 20 cm using 5 V supply. It was noticed during testing that the gain and cut-off threshold were calibrated well. For example, the threshold was too high at far distances and pulses were only partially detected. A digital potentiometer was added such that the threshold and gain can be adjusted. Hence, the final design is Figure 2(b) and Figure 3(b).

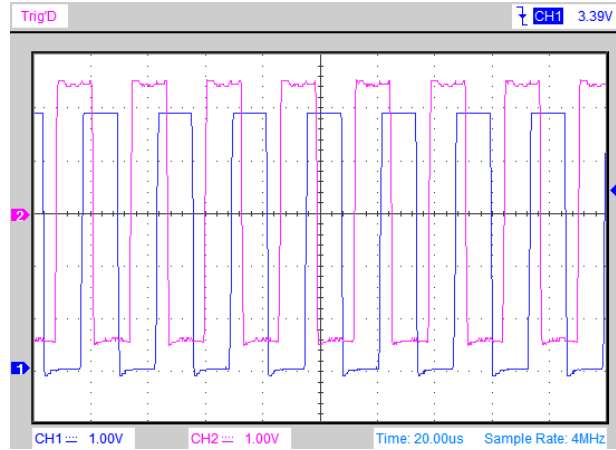


Figure 4: Transceiver Interface Testing at Distance of 20 cm

3.2 - Ultrasound & RF Multiple Access Control and Beacon/Receiver Synchronization

A combination of Ultrasonic and Radio Frequency signals will be used to determine the location of the receiver. In order for these signals to be useful in determining the location of the receiver, they must not interfere with each other as the receiver may not be able to distinguish which beacon generated the ultrasonic pulse or there may be collisions with multiple radio frequency signals, which would distort them. Multiple access control has to be taken into account as the Indoor Positioning System since it will not be reliable without it. Once the receiver has received 3 ultrasonic pulses from different beacons, it will apply a trilateration algorithm (using 3 distance measurements to calculate location) to determine its position within the map. An RF message will be used in combination with an ultrasonic pulse and the time difference of arrival between the 2 signals can be used to get a distance from the beacon to the receiver.

When determining how to perform multiple access control and beacon/receiver synchronization, several methods were taken into account. The methods analyzed are token passing, super-peer synchronization and CSMA/CA (carrier sense multiple access collision avoidance). In order to determine the optimal method, a decision matrix is used with weights assigned to criteria that are important in satisfying the specifications and the overall quality of the system. There are 4 criteria under consideration to determine the optimal method. These are simplicity, communication cost, scalability and accuracy. Each one is weighted at 25% as they are all equally important.

3.2.1 – Analysis of Strategies

Token Passing

For token passing, only the beacon that possesses the token (a simple software-implemented identifier) is allowed to transmit its ultrasonic signals. All other beacons have to wait until they receive the token to transmit. The token would be passed to other beacons in the RF message and piggy-backing will be performed by adding the ‘next beacon ID’ field in the data frame (a sequence of bits of fixed length) that is used in the trilateration process in order to reduce the number of RF packets sent. In order for the receiver to update its position 2 times/second, it will need to calculate its position every 0.5s. Given the range and width of the ultrasonic signals, it will take about 30ms for each of them to fully attenuate. This will allow for $500\text{ms}/(30\text{ms}/\text{transmission}) = 17$ transmissions within 0.5 seconds. This will allow for a system of 17 beacons, each with a unique ID.

The RF message that will be sent will have 5 fields, each of which is 1 byte long. These are two preamble flag fields, 1 byte to denote the next beacon that will transmit, 1 byte for the beacon that is transmitting and 1 byte for a checksum. The preamble flag fields represent the start of a message. Two preamble flag fields have been chosen because the communication channel has noise which can perturb the

message. In the worst case, it is assumed that the receiver is 10 m away from the beacon that is transmitting. The RF message will effectively be received instantaneously since it travels at the speed of light and the ultrasonic signal will be received about $10 \text{ m} / (343 \text{ m/s}) = 0.29 \text{ seconds}$ later since the speed of sound is about 343 m/s. Within these 0.29 seconds, the clocks may differ (skew) by at most $0.29 * (1 + 20 * 10^{-6}) - (1 - 20 * 10^{-6}) = 0.29 * 40 * 10^{-6} \text{ seconds} = \frac{1}{857500} \text{ seconds}$. This skew is very small and will lead to a position error of at most $1/857500 \text{ s} * 343 \text{ m/s} = 0.0004 \text{ m or } 0.04 \text{ cm}$. This system is highly accurate and satisfies the specification that the system should monitor a stationary receiver within 8 cm.

One of the issues with having a single token is that in a large enough space, the token may be passed around several times before an ultrasonic signal is received by the receiver. The beacons can transmit at a maximum of 10 m and, if the robot and beacon which holds the token are on opposite ends of a big space, the robot will not receive the ultrasonic signals. A workaround for this could be to have the beacons organized into groups and each group can have its own token. The packet format would now need to contain a group ID so that the receiver knows which group the ultrasonic signal is coming from.

Another issue with this mechanism is that there is a single point of failure. If the beacon that contains the token dies prior to passing the token onto the next beacon, the token is lost and, as a result, the receiver will not be able to update its position. One of the ways to handle this is to have a timer in each beacon that counts how long it has been since it has had the token. Once this value reaches 500 ms + the clock skew within this time = 500.02 ms, the beacon knows that it should have received the token and, as a result, will assume ownership of the token.

This system is fairly simple, as highly repetitive tasks are performed and beacons are told exactly when they should transmit their ultrasonic signal. It does, however, incur a high communication cost as there are 34 RF messages transmitted every second and each of these messages contain 6 bytes. This accumulates to 34 messages/minute*6 bytes/message = 204 bytes/minute. Each transmission requires power, so the token passing method wastes a lot of power in communication. The system is also scalable as the beacon that holds the token can die but the system will still work.

Super-Peer Synchronization

Another alternative is to have a super-peer (a node in a network that directs other nodes) that performs the clock synchronization. As stated above, there can be 17 ultrasonic and RF transmissions within 500 ms in order to achieve 2 position updates/second. In this scenario, a super-peer is responsible for 17 other beacons and transmits an RF message often enough to notify beacons that they should reset their clocks to mitigate the clock skew. The beacons will not need to send RF messages because the receiver and beacons have the same concept of time and updates will be performed frequently enough.

Preliminary tests show that the clocks operate at $20 \text{ MHz} \pm 400 \text{ Hz}$, the receiver is the slowest clock, and one of the normal positioning beacons will have the fastest clock. The difference between these two clocks after 1 second is $(1 + 20 * 10^{-6}) - (1 - 20 * 10^{-6}) = 40 * 10^{-6} \text{ seconds}$. Assuming that the speed of ultrasonic waves are 343 m/s, the maximum allowed skew can be calculated by assuming the worst case scenario that still meets the specifications. In this case, it would be an accuracy of 8 cm. The maximum allowed skew is $343 \text{ m/s} * t = 0.08 \text{ m}, t = 1/4287.5 \text{ second}$. The clocks will vary by at most $40 * 10^{-6} \text{ seconds}$ after 1 second, so in order for them to vary by $1/4287.5 \text{ second}$, it will take $1/(40 * 10^{-6}) * 1/4287.5 = 5.83 \text{ seconds}$. Therefore, in the worst case, clocks must be synchronized every 5.83 seconds. Synchronization will be performed more frequently to reduce the error in the position of the receiver. The value chosen is every 1 second and it will introduce a maximum error of $(40 * 10^{-6} * 1 \text{ seconds}) * 343 \text{ m/s} = 1.375 \text{ cm}$. This system is fairly accurate but does not offer the level of precision that the token passing method offers. In regards to the specification that refers to the accuracy of a moving robot, it is assumed that the robot moves at a maximum speed of 10 cm /s and its position is updated at 2 Hz. In order

to analyze this scenario, assume that the 3 points used in the trilateration calculation are received at 30ms, 240ms and 480ms. The radius of the robot's position will be known within 30ms. Since the next sample is received at 240ms, the robot will have travelled at most $10\text{cm/s} \cdot (240\text{ms} - 30\text{ms}) / (1000\text{ms/s}) = 2.1\text{cm}$. When the 3rd ultrasonic sample is received, the robot will have travelled another $10\text{cm/s} \cdot (480\text{ms} - 240\text{ms}) = 2.4\text{cm}$. During this time, if the robot would have travelled in one direction $2.4\text{cm} + 2.1\text{cm} = 4.5\text{cm}$, but the trilateration methods may have calculated that the robot travelled $2.4\text{cm} + 2.1\text{cm}$ in the other direction. Here, there would be a discrepancy of $4.5\text{cm} - (-4.5\text{cm}) = 9\text{cm}$. This satisfies the specification that there must be an accuracy of $\pm 15\text{cm}$ when the robot is moving at 10 cm/s .

One of the pitfalls with the above method is that there is a single point of failure because if the super-peer dies, there is no coordination and the beacons will not transmit. If this does happen, an election will take place to determine the next super-peer [10]. A beacon will detect that it has not received the clock synchronization in 1.00004 seconds and will begin by sending an election message to its neighbor, where its neighbor will be the beacon with ID $(MY_ID + 1) \% 18$. Since all beacons know which beacon is the super-peer that died, if $(MY_ID + 1) \% 18$ is equal to the ID of the super-peer then it will send to the beacon with ID $(MY_ID + 2) \% 18$. The value 1.00004 seconds is chosen because of the difference in clocks and since synchronization is supposed to occur every second, the maximum difference between 2 clocks in 1 second will be 0.00004 seconds. The neighbor that received the initial start of election message will add its own ID to the message and pass it along to its neighbor and this process will continue until the message comes back to the initiator of the election. At this point, the initiator will perform a linear search of the IDs and pick the beacon with the highest ID as the new coordinator. It will then pass a coordinator message around which will contain the ID of the peer that has been elected as the super-peer. The initial election message will contain 6 bytes, 2 for the preamble flags, 1 for length, 1 for the beacon ID, 1 for the ID of the neighbor that it is transmitting to and 1 for a checksum. As the election messages are passed around, each beacon will place its ID into the message which will increase the size by 1 byte.

An issue with this election mechanism is that there can be multiple elections that start at approximately the same time. The beacons would be receiving multiple RF messages at once and the messages would interfere with each other and potentially corrupt each other. This can be resolved by performing something similar to CSMA/CA (carrier sense multiple access collision avoidance) where the beacons wait and listen for a duration of time prior to transmitting any RF messages. This duration of time would have to be long enough to accommodate for the propagation and transmission of the RF message. Since the initial election message contains 6 bytes, and the RF modules have a BAUD rate of 4800 bits/second, it will take a total time of $t = \frac{6\text{ bytes} \cdot 8\text{ bits/byte}}{4800\text{ bits/second}} = \frac{1}{100}\text{ seconds}$ assuming that the propagation is negligible since the message travels at the speed of light. Once a beacon's timer goes off for it to start an election, it will wait and listen for any RF transmissions for $1/100$ seconds if it has not already received an election message. Under these conditions, there will be no issue of having election messages interfere with each other and the election can proceed without issues.

In order to make the system scalable, several modifications must be made. Firstly, it does not account for large areas with just 1 super-peer and 17 beacons. More beacons and super-peers must be added to the system in large spaces and beacons can be organized into groups, where groups consist of 17 transmitters and 1 super-peer. Now that there are multiple groups, the robot must be able to distinguish the RF messages between the super-peers. This can be solved by adding a byte to the message that corresponds to the group ID of the super-peer. The super-peer is aware of all of the nodes with which it must perform synchronization. This will allow it to send a packet that contains a mapping of IDs to when a beacon should transmit. Because every beacon must transmit once within the 500ms interval required for a position update and knowing that the ultrasonic transmissions must be spaced apart by 30ms, the data can be encoded in

such a way where it takes $\text{Ceil}(\log_2(17)) = 5$ bits to represent when each beacon should transmit, and there are 17 beacons hence $5 \text{ bytes/beacon} * 17 \text{ beacons} = 75 \text{ bytes}$ are needed. For example, if the message 00011 01111 01011 00000 is transmitted, then the beacon with ID 2 will transmit at time $01011 * 30\text{ms} = 330\text{ms}$. At this point, the system is highly scalable as the super-peers can die and the system will still work and the beacons are organized into physical groups in order to cover a larger area. In order to account for the insertion of a beacon, if a beacon is not associated with a super-peer, it will broadcast a request to join a group message. A super-peer will respond by giving it an ID between [0, 17] that is not currently being used. This makes for insertion of beacons into the system scalable and does not require the user to perform programming.

The receiver has the same concept of time as the beacons and will store the RF broadcast messages that the super-peers send out. Once this message is received, the receiver will process it so that it can sort the IDs of the beacons by the time that they will transmit as it will make it easier to determine what beacon has transmitted. Once ultrasonic pulses are received, the receiver will be able to perform the trilateration algorithms to determine its position. If no ultrasonic pulse is received within 30ms, the receiver will assume that the beacon has transmitted but it was unable to receive the signal. The format of the RF message that the super-peer will send out is 2 bytes for the preamble flag, 1 byte for the group ID flag, 10 bytes for the beacon transmit times and 1 byte for the checksum. The total communication cost for this system is 14 bytes/second as the frame is sent out once per second. This is a relatively low communication cost and only 1 beacon within each group will be performing this transmission.

There are issues if 2 beacons are within transmission range of each other and the robot happens to be positioned within that range. If both beacons transmit at the same time, the robot will not know which beacon sent the ultrasonic signal since it cannot determine which group the beacon that transmitted belongs to. In order to minimize this issue, the receiver must keep track of which group it was previously in. The robot will be moving at a maximum speed of 10cm/s and will therefore move at most $30\text{ms} * 10\text{cm/s} = 0.3\text{cm}$ between the time 2 ultrasonic pulses are transmitted. Once it enters a region that 2 groups cover, it will receive 2 ultrasonic pulses in less than 30ms and can use the difference in these times to determine where within this boundary it is located. If the difference between times of 2 ultrasonic pulses decreases, this information can be used to help determine its position and it can be assumed that the robot is moving closer to the middle of the boundary. Once the differences in time between the 2 ultrasonic signals increases after decreasing, it will be assumed that the robot is entering the region for the next group. Once the robot is out of this boundary and into an area that only 1 group covers, it can determine its exact position as it will only receive 1 ultrasonic pulse within the 30ms duration. If the assumption that the robot was exiting the current group and entering the next is incorrect, it will be reflected when the robot transitions from the boundary into a region that is solely covered by one group.

This system is fairly complex in terms of the amount of computation the receiver performs. Once the RF messages are received, it will need to process them and order the IDs by time to transmit. It is also fairly complex in how the receiver handles situations where it is in a boundary at which point it will receive multiple RF messages from different super-peers. It will also need to determine the difference in arrival time between the ultrasonic transmissions by beacons in different groups to determine whether it is crossing the boundary and entering a different group. The system also requires a lot of coordination and message sending when an election is occurring after the super-peer has died.

CSMA/CA

Another design alternative is to have the beacons wait and listen for a designated time interval prior to transmitting. If the beacon does not detect another beacon's RF signal while listening, it is allowed to transmit its signals. If the beacon does detect another beacon's RF signal, it will wait a random amount of

time before once again waiting for the designated time interval. RF signals are chosen instead of ultrasonic signals since the RF signals are omnidirectional whereas the ultrasonic signals are directed towards the receiver, which will be on the ground. This would be similar to CSMA/CA (carrier sense multiple access with collision avoidance) which is used in wireless networks [11]. The designated time interval would be large enough to account for how long the ultrasonic signal would be detectable by the receiver after its transmission. This time interval is chosen to reduce the number of collisions while attempting to ensure a high frequency for number of position updates by the receiver. If the time interval were shorter, there would be an increase in the number of collisions as there could be ultrasonic signals that have not fully died down that would interfere with other beacons' ultrasonic signals. If the designated time interval were increased, the receiver would receive less signals which decreases the number of updates it can perform.

One of the problems with this approach is that a beacon may not be able to detect another beacon's RF signal that is interfering with the current beacons RF signal if the beacons are too far apart. This is known as the hidden terminal problem and could cause 2 beacons to have their ultrasonic signals in the air at the same time, thereby interfering with one another. It is very difficult to work around this problem as it becomes difficult to distinguish which ultrasonic pulse corresponds to which beacon in the event that 2 RF messages are transmitted within the 30ms interval. There is no quality solution to this problem.

This system is similar to token passing in terms of the format of the RF message that is being transmitted except that the ID for the next beacon that is to be transmitted does not need to be included since the beacons will determine transmission time without any direct coordination. There will be 4 bytes, 2 preamble Flag bytes, 1 beacon ID byte and 1 checksum byte. It also exhibits similar accuracy measurements for a stationary receiver. Since there are 4 bytes and there will be at most 34 transmissions per second which will result in $4\text{bytes/transmission} * 34\text{ transmission/second} = 136\text{ bytes/second}$.

One of the benefits of using this method compared to the other 2 is that there is no single point of failure. If one of the beacons goes down, the remaining beacons can still function and transmit their signals. Another benefit with this method is that beacons do not need to be organized into groups. In the previous solutions, beacons that are within transmission range of each other but in different groups may have transmitted at the same time and the robot would not know the source of the ultrasonic transmission. This issue is removed as groups do not exist and beacons will not transmit if another beacon within its range is transmitting. Insertion of beacons into the system is very easy as the no coordination needs to be performed when a beacon is inserted and it can begin transmitting its ultrasonic signals.

This system is not very complicated as all that needs to be done is listening for RF messages which is followed by either more waiting or transmission of its own RF and ultrasonic signals depending on whether or not any RF or transmission occurred within the listening interval.

3.2.2 – Results of Analysis

Table 3 provides a summary of the analysis as a decision matrix. The final design chosen after considering all 3 is the super-peer synchronization method. In terms of simplicity, the token passing method is favourable while the super-peer method is the most difficult. The token passing method requires little coordination and uses the transmission of the token to synchronize clocks. As for communication cost, the super-peer method was far cheaper than the other 2 methods as it would transmit much fewer bytes/second over RF. As for scalability, they all work well as they handle the insertion and removal of beacons. For accuracy, the super-peer method has a mechanism to handle when the robot is in the boundaries of 2 groups of beacons because it contains the time at which it expects beacons to transmit. The other 2 methods do not have this and there aren't any elegant or accurate ways to work around this. CSMA/CA was the worst method for accuracy as it does not account for the hidden terminal problem, but it does achieve similar

accuracy as token passing when there are no collisions. Even though the super-peer method has slightly worse accuracy for a stationary object, the accuracy is still much less than that required of the specification.

Table 3: Summary of Beacon Synchronization and Multiple Access Control Methods

	Token Passing	Super-peer	CSMA/CA
Simplicity	25	15	20
Communication cost	15	25	15
Scalability	25	25	25
Accuracy	20	25	15
Total	80	90	75

3.3 - Speed of Sound Detector

3.3.1 - Initial Approach

As discussed in Section 3.2, it is necessary to know the speed of sound in order to use the time-of-arrival approach to positioning. The initially strategy was to calibrate this for a given environment at the time in which the positions of the beacons would be programmed. Thus the receivers and beacons would permanently store the speed of sound in their external EEPROM memory. It was assumed that an indoor location would have stable environmental conditions due to air conditioning systems. Experiments using the indoor positioning hardware showed that that this was false due to changing error over time. The speed of sound is known to increase with increasing temperature due to excitement of air molecules, and to a lesser extent with increasing humidity with the replacement of N_2 and O_2 molecules with lighter H_2O [12]. According to a model described in a paper [13] and implemented as a calculator [14], at 20 °C and 0% relative humidity (RH), changing pressure from 70 kPa to 102 kPa increases speed by only 0.01 m/s, while at 101.3 kPa, changing RH from 0% to 100% causes a change of 1.25 m/s. Temperature causes a change of $0.6 \text{ ms}^{-1}^\circ\text{C}$, and so a change from 20 °C to 30 °C (at 0% RH) would cause a change from 343.36 m/s to 346.27 m/s. With a Receiver 4 m from a Beacon, this causes a difference of 6.63 cm. Clearly, indoor temperatures vary over time due to delays in the feedback loops of heating systems, and some would not control for humidity (or pressure). Therefore, a means of measuring the speed of sound is required.

3.3.2 - General Strategies for Estimating the Speed of Sound

Three general strategies exist for measuring the speed of sound. Firstly, it is possible to measure the speed of sound directly by measuring the time it takes for sound to travel a known distance. Thus, this could be accomplished by placing an ultrasound receiver at a fixed distance from a transmitter and measuring the difference in time between transmission and reception. This is possible since electrical transmission is roughly six orders of magnitude faster. A second strategy would be to indirectly measure the speed of sound using a receiver and transmitter at fixed distances apart but rather than measuring the time difference, a sine wave could be frequency modulated such that wavelength would return to a fixed value and a node would appear at the position of the receiver. Since $v = \lambda f$ and nodes appear at $d = \lambda(2n + 1)/2, n \in \mathbb{Z}$. A third strategy would be to estimate the speed of sound with mathematical models using measurements of temperature and humidity.

3.3.3 - Analysis of Strategies

It is apparent that the advantage of direct measurement is that it would not require the calibration of sensors aside from an oscillator, nor would it be limited by the precision of a mathematical model. The obvious disadvantage is that it would introduce a higher system complexity than simply using a temperature and humidity sensor. For example, it would require similar, but dedicated interface circuitry as that described in Section 3.1 to receive and transmit a pulse. This would need to be present on each Receiver, and because the transducers would need to be at least several wavelengths apart, it would likely violate the volume specification for the receiver and increase the cost of the prototype substantially. It is also worth

noting that the use of ultrasound for speed measurement could interfere with receiver operation and may not be possible while the beacons and receivers are operating.

Indirect measurement would face similar physical, cost and complexity as direct measurement, but measurement through detection of nodes would prove less accurate due to the effects of background noise and the inability to differentiate between adjacent nodes. For example, if $v_1 = 434 \frac{m}{s}$, a node is expected at $d = 19 \frac{\lambda_1}{2} = 10.3 \text{ cm}$ with $f = 40 \text{ Hz}$ but if $17 \frac{\lambda_2}{2} = d$ and $f = 40 \text{ Hz}$ then $v_2 = 485 \text{ m/s}$. This shows that some other combination of frequency and speed may result in a node but with the wrong wavelength. Furthermore, the ultrasonic transducers have narrow bandwidths, which could limit adjustment. Hence, this method is possible but complex and likely unreliable. It will not be considered further.

Temperature and humidity sensors have the principal advantages of small size, low complexity, high viability, and low cost in comparison to the previous methods. However, it is obvious that sensors may not be calibrated correctly, have limited accuracy and resolution, and may perform more poorly over time. The first two devices considered are the DHT-11 and DHT-22 which measure temperature and humidity with $\pm 2^\circ\text{C}$ and $\pm 5\%RH$, and $\pm 0.2^\circ\text{C}$ and $\pm 2\%RH$ respectively. If one assumes an uncertainty of $0.606 \text{ ms}^{-1}/^\circ\text{C} + 0.0125 \text{ ms}^{-1}/\%RH$. Then the DHT-11 and DHT-22 have uncertainties of 1.3 m/s and 0.15 m/s respectively. The DS1624 is a temperature sensor with an accuracy of $\pm 0.5^\circ\text{C}$ which equates to $\pm 1.6 \text{ m/s}$. With the addition of the humidity sensor HIH4030 with an accuracy of $\pm 3.5\%RH$, this becomes 0.34 m/s. In terms of complexity, the DS1624 is the simplest device since it uses a standard I²C interface, in comparison to the DHT-11/22, which use a non-standard single wire serial interface. The HIH4030 requires a 5 V supply and uses an analog interface; thus, it adds reasonable amount of complexity beyond only the DS1624.

3.3.4 - Results of Analysis

Table 4 provides a summary of the previous analysis as decision matrix. The cost estimate are based on normalization of raw prices in the case of the sensors. For the direct method, other components such as op-amps are accounted for. Accuracy is estimated to be maximal for the direct approach, where for the others, it is calculated based on the uncertainty in terms of the speed of sound uncertainty due to measurement uncertainty. Complexity and Viability are qualitative values based on the previous discussion. In conclusion, the chosen design is to use a mathematical model for the relation of temperature and humidity to the speed of sound and the DHT-22 as a sensor. In theory, this leads to distance uncertainty of $\pm 1.4 \text{ mm}$ at 4 m from a beacon and a speed of 443 m/s. Thus the contributed position uncertainty due to changing speed should be far lower than the $\pm 8 \text{ cm}$ specification.

Table 4: Summary of Speed of Sound Detection Methods

	Direct		DHT-11		DHT-22		DS1624		DS1624+HIH4030	
Cost (20%)	8\$	78%	2\$	94%	4\$	89%	4\$	89%	18\$	50%
Accuracy (35%)	100%		$\pm 1.3\text{m/s}$	62%	$\pm 0.15\text{m/s}$	96%	$\pm 1.6\text{m/s}$	53%	$\pm 0.34\text{m/s}$	91%
Complexity (20%)	30%		90%		90%		100%		70%	
Viability (25%)	50%		100%		100%		100%		90%	
	69%		84%		94%		81%		78%	

3.4 - Navigation & Path-Finding System

3.4.1 - Representation of Physical Space

The coordinate system and representation of the physical space in which the robot will inhabit must satisfy the several key requirements for it to perform navigation. Firstly, it must represent the position of the robot. Secondly, it must provide a mutable representation of the map such that the obstacles may be learned and plotted as encountered or shared between robots through the robot control interface. Thirdly,

the position of the goal location must be represented, and fourthly, the position of the next move. Two schemes are presented.

Cell-Based Scheme

In a cell-based approach, the area of the map will be discretized into distinct cells of some fixed length and height. Each of the cells will store three values that will be used to aid the path-planning algorithm. The values are whether the cell contains an object, whether the cell has been visited during the current path, and whether the cell is unexplored. If an obstacle consumes more than half the area of a cell then the entire cell is considered to be occupied. Maneuvering the robot would be performed on a cell-by-cell basis such that the vehicle will always move to a neighboring cell. Using the IPS Receiver the navigation system would determine the robot's position and the cell it inhabits. With the current cell, adjacent cells can be determined such that the next move can be chosen from one of these cells. The goal would similarly be represented as a cell; hence, navigation completes on reaching this cell.

Floating-Point Scheme

Define the coordinate system using two floating-point numbers which correspond to a unit of distance such as metres. These numbers represent x, y coordinates on a 2D plane. Objects within this space will be represented as rectangle defined by ranges of x and y values. For example an object may be represented as $x = [3, 4]$ and $y = [2.5, 3.5]$ which would be a representation of a square obstacle. The robot's center point would represent the robot's position, for example $[1,1]$. The goal position would be represented similarly. The goal is reached when the Euclidean distance between the goal and robot is below a threshold.

Each method is evaluated on its accuracy in the modelling of real world positions and how well it can represent a graph (a set of connected nodes and edges) and easily be adapted for well-known path finding algorithms. Table 5 provides a summary of this analysis.

Table 5: Evaluation of Map Representation Schemes for Navigation

	Graph Representation Simplicity	Position Modelling Accuracy
Cell-Based Scheme	Physical space can easily be separated into nodes or cells of a discrete width and height. Obstacles can be represented as filled cells and a graph can be drawn (for path-planning use) such that adjacent cells and nodes are connected. Known path finding algorithms typically are used with connected graphs. Ability to build or modify a map after finding an obstacle is easy since it requires only marking a cell as occupied.	The resolution of cells determines how accurately the coordinates will be represented in terms of physical units since an object may not be centered within a cell; neither will it necessarily occupy the entire cell area.
Floating-Point Scheme	Floating-point precision is a problem when determining spanning area of an obstacle and whether the robot will be attempting to enter an obstacle. It is necessary to check ranges of values in order to determine where obstacles are. Thus, updating a map when discovering obstacles becomes much more complicated. A general approach might be to use a road-map algorithm to periodically redistribute node positions by randomly placing nodes on the map, removing nodes on or near obstacles and then clustering nodes and placing edges between adjacent clusters. This would be complex and computationally expensive.	It is possible to represent nearly precise positions in the coordinate space; however, since objects are represented as rectangles, round obstacles become more complicated to represent precisely. Similarly coalescing smaller obstacles is necessary to reduce computational cost, but this operation is costly itself.

The chosen design uses a cell-based method, which meets the need to easily provide a representation of a connected graph. Implementation of a floating-point scheme would be problematic in terms of defining a connected graph and representing non-rectangular objects. Building a graph in a continuous space and adapting as new obstacles would be complex and expensive. Furthermore it would be necessary to modify or merge obstacles as more information is discovered to reduce computation and storage cost. However, floating-point coordinates clearly offer greater position precision, although a cell-based approach should be sufficient if the cells have adequately small physical area but sufficiently large enough to avoid excessive memory and computation requirements with a huge graph.

3.4.2 - Path-Finding Algorithms

The robot provides limited computational resources and sensing capabilities to the path-finding algorithm. These include a limited distance in which it is able to detect obstacles with infrared sensors, and the limited accuracy and precision afforded by the indoor positioning system receiver. Computationally, the Raspberry PI is much less capable than a computer. In order to critically evaluate and compare the proposed algorithms, a set of evaluation criteria must be defined. These are summarized in Table 6.

Table 6: Navigation Algorithm Analysis Metrics

Metric	Description
Computation Cost	Latency between computation of the best path and moving the robot around must be minimized in order to produce a seamless path finding algorithm for the robot. This is computation cost per move in the cell space. The analysis differentiates the cases when there are obstacles and when there are not.
Simulation Time: Unknown Map	This refers to how long it takes the robot to get from its location to the goal without any prior knowledge of the map. Simulation run time includes only the time for robot motion and does not include the time the program takes in order to find the best path.
Simulation Time: Partial Map	This refers to how long it takes the robot to get from the goal location to its original start location, only knowing the obstacles it encountered on the first-pass. Thus this measures the extent to which learning the map improves the optimality of the path.

A* Search Algorithm

A* is a standard path finding algorithm which operates similar to Dijkstra; however it is an informed search and node selection changes based on how close the node are to the goal. It can be used in several different ways to find the shortest path from robot's current position to its destination. Initially, the robot will start in a known location and it knows the destination cell (the A* heuristics require this), but it will not know where the obstacles are positioned. A* will find the shortest path; however, it may encounter an obstacle that it does not know of. In this case, the map is updated and the algorithm is rerun in order to calculate a new shortest-path based on the new data. It is notable that this is not directly the intended use of A*, but it was found to be useful when encountering new obstacles. This process will continue until the robot reaches the goal. Hence, A* is more likely to find better paths if it knows more about the map.

Best-First Search Algorithm

Best-First is a greedy algorithm, which will always visit the neighboring node with the least cost value. The chosen heuristic is the distance from the current cell to the goal cell. This algorithm works with the cell-based map by calculating the heuristic for the cells surrounding the robot's current location. The robot then chooses the cell with the least distance to the goal. Note that traversed cells are not considered neighbors. While moving, if a wall is detected using the infrared sensors on the front of the robot, then the robot would stop, recalculate its current cell, record the presence of an obstacle in that cell, and find the next best cell to move to. Best-first suffers from entrapment in local minima where there are no adjacent cells for which it has not traversed. Should this occur, the algorithm directs the robot along its previously traversed path until it is able to find cells for which it can continue to progress towards the goal.

Potential Field

Potential field is similar to Best-First in that it will check all cells surrounding the current cell and then choose the best option. The main difference is how it calculates the cost for these cells. For potential field the cost of all cells is assigned before beginning to move towards the goal. The costs of cells for potential field are determined by the distance from the goal as well as the distance from obstacles. Cost improves when closer to the goal and worsens when closer to obstacles. After assigning a value to each cell, a path can be created by greedily choosing the lowest cost cells around the current position. Similarly to Best-First, this has a potential to reach a local minima in which backtracking will occur. Learning is involved in this algorithm and when a new obstacle is found the costs for cells around the obstacle are recalculated.

Since the algorithm will choose the least cost cell and cells closer to objects are of higher value, this algorithm will smoothly curve around obstacles and towards the goal.

Trace

Trace is a custom algorithm. Trace's use is intended to trace out objects, perhaps used by the robot in a learning phase in order to find the obstacles in a map. Trace will journey towards the goal, and when it encounters a wall it tries to follow that wall until it circumnavigates it or until it reaches the edge of the map. This method is broken up into several steps. The first step is a Best-first search that runs until an object is encountered. It will then check to see if the obstacle is a lone object or a wall. A wall is denoted as 3 parallel cells being filled with an object. If it is a wall, then the algorithm switches to phase two. It begins by choosing a direction represented by two numbers, each of which is in the range of $[-1, 1]$. For example $(-1, 0)$ would represent the cell directly north of the current cell. After finding the direction, it then starts a best-first search in that direction. This means that instead of evaluating all eight cells around the current cell, the algorithm need only look at 3 in the direction of movement. Trace continues doing this until the wall is no longer adjacent or until another wall appears in the new direction. If the wall ends, it goes back to the Best-First search, otherwise if there is a second wall it determines what direction to travel next. The Trace simulation uses Best-First when it is not tracing an obstacle. The Best-first algorithm can be replaced by an A* algorithm for path recalculations after circumnavigating an obstacle.

3.4.3 - Simulation and Testing

In order to provide quantifiable data to determine how well these algorithms would perform in practice, a discrete-time path finding simulator was created in Java. It models the speed of a robot and discretizes the maps using a cell-based representation of the physical space. The four algorithms were implemented and tested with the maps listed in Table 7.

Table 7: Navigation Algorithm Simulator Maps

Map 0	Large obstacle with small opening which can leads to a path to the goal
Map 1	A simple straight line between the start and goal locations
Map 2	A horseshoe shape around the start location. Goal is located outside the horseshoe on the closed side.
Map 3	A spiral maze with the goal in the middle of the spiral and the start on the outside of it.
Map 4	An obstacle almost completely trapping the start location with the edge of the map and the object
Map 5	A circular object in the direct path from start to goal location
Map 6	No objects just the simple start and goal location

First-Pass Run Times

Table 8 shows run times for navigating through the seven maps with no prior information about map topology. All maps have the same start and end locations and thus the same direct distance between start and goal location. It is permissible for the algorithm to store obstacles that it finds on its path to the goal such that it may learn the map topology.

Table 8: Navigation Algorithm Simulation Times – First Pass

Algorithm	Map 0	Map 1	Map 2	Map 3	Map 4	Map 5	Map 6	Sum
A*	6.59 s	5.54 s	3.95 s	16.97 s	10.25 s	3.77 s	2.98 s	47.08 s
Best-First	16.76 s	7.24 s	3.88 s	121.69 s	30.66 s	3.70 s	2.98 s	183.96 s
Potential Field	22.78 s	5.90 s	4.32 s	145.41 s	37.96 s	3.70 s	2.98 s	220.06 s
Trace	5.58 s	5.80 s	3.88 s	17.10 s	10.50 s	3.70 s	2.98 s	49.54 s

Second-Pass Run Times

The robot is designed to work in an office environment with obstacles that do not vary greatly. When performing path planning the algorithms are able to dynamically update the map in order to obstacles that

have been found on its previous journeys through the area. With this information it is possible to use the learnt map subset to evaluate a possibly lower cost path from start to goal. In certain algorithms, knowing the map beforehand can lead to lower cost paths. Table 9 shows the results of the simulator when a partially known map is used.

Table 9: Navigation Algorithm Simulation Times – Second Pass

Algorithm	Map 0	Map 1	Map 2	Map 3	Map 4	Map 5	Sum
A*	6.08 s	3.58 s	5.92 s	14.32 s	10.44 s	2.92 s	43.26 s
Best-First	16.77 s	7.25 s	3.89 s	121.70 s	30.67 s	3.71 s	183.99 s
Potential Field	15.62 s	4.86 s	7.23 s	115.02 s	11.08 s	7.51 s	161.33 s
Trace	5.58 s	5.80 s	3.88 s	17.10 s	10.50 s	3.70 s	46.56 s

3.4.4 - Computation Cost Analysis

Table 9 provides a summary of the expected computational costs for choosing the next cell. That is, the cost of online computation rather than offline planning that might be performed at the start. Costs may vary substantially based on whether a new obstacle is discovered when searching for the next move or whether it may simply proceed as planned.

Table 10: Analysis of Computation Cost per Cell Move in Robot Navigation

Algorithm	Computation Cost Per Cell Move (no obstacle)	Computation Cost Per Cell Move (obstacle found, or initial calculation)
A*	Only need to calculate the least cost path initially than follow path till the goal as long as no unexpected obstacles are found. When simply following the planned path there is a pre-found cell to go. Hence cost is $O(1)$.	When an obstacle is found, the least-cost path must be recalculated with the updated map. A* is in the worst case, exponential in complexity; however, if there is a single goal state and the heuristic chosen is the distance from the cell to the goal cell, then A* complexity becomes polynomial.
Best-First	It needs to only calculate the heuristic value of at most 8 surrounding cells, which has a constant time $O(1)$ cost.	Best-First has the same computation cost as without an obstacle plus the cost of updating the map. Thus, it still constant time $O(1)$.
Potential Field	It simply follows the cell with the best heuristic value. This is constant and at most 8 cells evaluated. Hence, $O(1)$.	Need to recalculate potential field values for cells around obstacle and perhaps the entire $N \times M$ grid space. Thus, $O(NM)$.
Trace	It is a combination of best-first and moving along a wall. $O(1)$ since it needs to compute at most the costs of 8 cells.	Cost is identical to the case with no obstacle plus updating the map to include the new obstacle. The computational cost is still constant time $O(1)$.

3.4.5 - Results of Analysis

Based on the decision matrix in Table 11, it was found that the best algorithm for use under for robot navigation under the given circumstances is A*. Overall this algorithm appears best meet the performance metrics for a known or an unknown map. It should be noted that A* may improve or decrease in performance depending on the map and the available partial knowledge of the map. When avoiding known objects A* may head into an unexplored part of the map and end up in an area with no short path to the goal. This causes it to recalculate and likely backtrack along its path. The more of the map A* knows the shorter its path will and it will actively seek the shortest path.

Table 11: Decision Matrix for Navigation Algorithm

Metric	A*	Best-First	Potential Field	Trace
Simulation Run Time Unknown Map (30%)	100	20	10	90
Simulation Run Time Partially Known Map (45%)	100	20	30	80
Computation Cost Per Move: No obstacles (10%)	100	100	100	100
Computation Cost Per Move: Obstacles (15%)	30	100	50	100
Summation	89.5	40	34	88

3.4.6 - Integration with Physical Robot

The final design uses IPS and infrared sensors on the front of the robot in order to allow the robot to move from a source to a given destination within the bounds of the IPS. The robot will start knowing the position of itself (through IPS) as well as the position of the goal cell. The robot will then calculate the least cost path using A* from itself to the goal cell and follow this path. The path will be a list of cells which the robot will traverse sequentially. The robot initially stores a map as a grid of cells in which there are no obstacles occupying any of the cells. Once the robot detects obstacle in a cell in its path, it will recalculate the least-cost path from its current location to the goal; however, it will first mark the cell as occupied in its map. The A* algorithm will now know to avoid this cell, and will compute a more correct path to the goal, if another obstacle is found a similar process will occur until the goal is reached.

3.5 - Robot Control System

This section describes the hardware and embedded system design used in the robot prototype.

3.5.1 – Robot Circuitry and Part Selection

Figure 5 below shows a schematic of the robot. Each component was specifically chosen to meet the specifications of the report and to safely meet the electrical requirements of each component. Note that the five 1.2 V AA batteries have a capacity of 2400 mAh. Since the robot will draw at most 1.5 A from the batteries during normal operation, this is enough to power the robot for at least an hour. So, the battery life specification is satisfied.

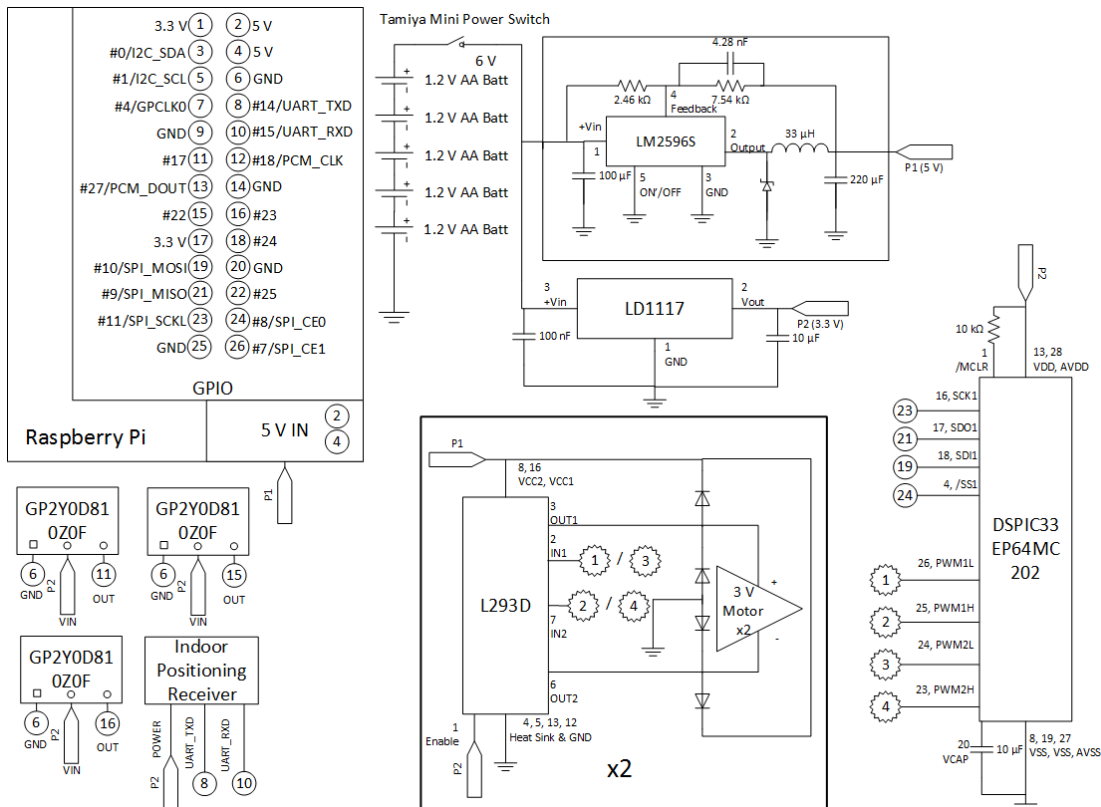


Figure 5: Robot Circuitry

The main controller for the robot is a Raspberry Pi. This is essentially a very small Linux based computer that will be used to perform all robot functions. The robot needs to communicate with a Graphical User Interface (GUI) at any given time, calculate a path to travel, and control the motors while watching out for obstacles and receiving location data from an ultrasonic positioning receiver. Since Bluetooth is often

inconvenient and communication via LAN requires prior knowledge of the device's local IP address or MAC address, a WIFI interface connecting to a separate server is the best option for communication with a GUI. The Raspberry Pi offers built-in USB ports for a WIFI dongle and it has sufficient processing power to handle all its other tasks.

Two voltage rails are shown in the diagram above. The 5 V rail is necessary to power the Raspberry Pi and motors. The 3.3 V rail is necessary for the infrared proximity sensors (GP2Y0D810Z0F), the indoor positioning receiver, the motor microcontroller (DSPIC33EP64MC202), and the motor driver (LD293D).

3.5.2 – Motor Control Modeling

The control for the velocity of the motor is implemented in the motor microcontroller (DSPIC33EP64MC202). The Raspberry Pi sends a desired angle to the microcontroller, which implements a digital control system to regulate motor rotation. This is accomplished via Pulse Width Modulation (PWM), which uses short 3.3 V DC pulses to increase or decrease the speed of the wheel. This requires a precise clock and constant attention from the microcontroller. Thus, it was not implemented in the Raspberry Pi since the Linux scheduler could ignore the PWM output while it is busy with other tasks. So, the control system will have a reference input from the Raspberry Pi (rad). Duty cycle refers to the percentage of the PWM period that the motor is driven. Thus, a 100% duty cycle implies full speed and a 0% duty cycle implies a full stop. This means that the plant in this system is a function of the duty cycle.

It should be noted that this controller will try to maintain single-directional movement at all times. Changing direction, such as when correcting for overshoot, is slow for the robot. Therefore, the desired position is interpreted as positive movement in a specified direction. As shown in Figure 5 above, there are two separate inputs in the motor driver (L293D) that are controlled by different PWM channels. Movement occurs when one, but not both, of those inputs is driven. Feedback for this system may come from sensors that approximate the angle of the wheels. This comes from two infrared sensors spaced 30° apart such that one sensor will trigger at a time while the spokes in the robot wheel are turning. In this way, the sensors can calculate the rotation of the wheel. However, this is mostly useful in error correction since the control system already calculates an approximate angle of rotation.

Some preliminary tests were conducted to determine the speed of the wheel at various duty cycles with “no load” (wheel spinning in the air). These are summarized in Table 12.

Table 12: Robot Speed Measurements

Measurement	Value	Measurement	Value
Wheel Diameter	0.062 m	50.0% Duty Cycle	0.262 m/s
0% Duty Cycle	0 m/s	62.5% Duty Cycle	0.345 m/s
12.5% Duty Cycle	0 m/s	75.0% Duty Cycle	0.384 m/s
25.0% Duty Cycle	0 m/s	87.5% Duty Cycle	0.478 m/s
37.5% Duty Cycle	0 m/s	100% Duty Cycle	0.531 m/s

Clearly, this is a saturated function limited by approximately 0.262 m/s and 0.531 m/s. In between these values, the relationship between duty cycle and speed is roughly linear. This is a decent approximation when the robot is travelling on surfaces that do not present significant resistance to movement, such as hardwood flooring. The model breaks down on surfaces such as thick, fluffy carpet. So, the duty cycle, $d(t) \in [0,100]$, is related to the speed of the robot, $v(t) \in [0.262,0.531]$, by $d(t) = \frac{v(t)}{5.31 \times 10^{-3}}$.

3.5.3 - Motor Controller Design

Now that each component has been identified and modeled, a simple control system may be implemented. One solution would be to simply assume feed forward control, but this would not provide very good tracking in a system with as many uncertainties as this one. Thus, a control system with a simple

feedback loop is used to control for movement whenever possible. The robot is initially at rest. The system is given by Figure 6 below. Here, r is the reference angle, e is the error in the angle, $C(s)$ is the controller, d is the duty cycle, $P(s)$ is the plant, and y is the output angle. The plant is given by the equations below. Note that $x(t)$ is the position of the robot.

$$d(t) = \frac{v(t)}{5.31 \times 10^{-3}} = \frac{1}{5.31 \times 10^{-3}} \frac{dx(t)}{dt} = \frac{0.062}{5.31 \times 10^{-3}} \frac{dy(t)}{dt}$$

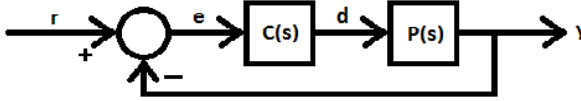
$$P(s) = \frac{Y(s)}{D(s)} = \frac{1}{11.676s}$$


Figure 6: Unity Feedback Motor Control System

Thus, the plant is critically stable. This is undesirable, so it is necessary for the controller to stabilize the system. A quick root locus plot, as shown in Figure 7 below shows that adding a pole in the open left hand plane (OHLP) can stabilize the system. Suppose the controller is given by $C(s) = \frac{K}{s+a}$ where K is the gain, a is the location of the pole, and $C(s)$ is implemented in software. Looking at Figure 7, it is clear that the system is stable for all $K > 0$ and $a > 0$. It is left is to determine K and n such that the robot will reach its destination quickly without having to backtrack and without using a duty cycle outside of [50, 100].

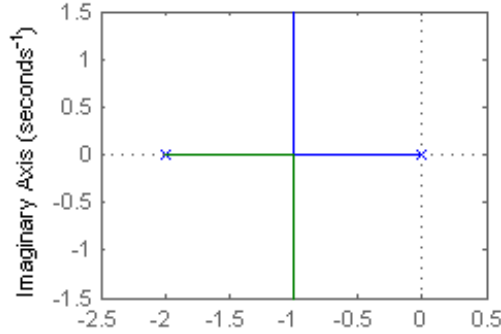


Figure 7: Root Locus Plot of Motor Control System

An iterative approach was taken to find a system with little to no overshoot where the slope of the curve indicates a speed that is viable for this robot at all times. Thus, the target system is an overdamped system or an underdamped system with sufficiently small overshoot such that the motor will not actually move. Note that the system is now a typical second order control system with transfer function $G(s)$ that can be written in standard form.

$$G(s) = \frac{P(s)C(s)}{1 + P(s)C(s)} = \frac{K}{11.676s^2 + 11.676as + K} = \frac{\frac{K}{11.676}}{s^2 + as + \frac{K}{11.676}} = \frac{\omega_n^2}{s^2 + 2\xi\omega_n s + \omega_n^2}$$

$$\omega_n^2 = \frac{K}{11.676}; 2\xi\omega_n = 2\xi\sqrt{\frac{K}{11.676}} = a \rightarrow \xi = \frac{a}{2}\sqrt{\frac{11.676}{K}}$$

Consider that the step response corresponds to forward movement by 6.2 cm and that the medium speed of the robot is approximately 0.384 m/s. Furthermore, the system will “settle” within larger than 2% of the final value since the PWM duty cycle will likely not be sufficient to drive the motor as the step response progresses in the time domain. Thus, the usual calculation for settling time will overestimate the actual settling time of this system. So, a reasonable starting point is calculated on the following page.

$$Target = \frac{0.062}{0.384} = 0.161 \text{ seconds} \rightarrow 2 \% \text{ Settling time} = T_s = \frac{4}{\xi \omega_n} \approx 2 * Target = 0.323 \text{ seconds}$$

$$Heavy Damping \rightarrow Pick \xi = 0.7; \omega_n = \frac{4}{\xi T_s} = 17.691$$

$$K = 11.676 \omega_n^2 = 3654.255; a = 2\xi \sqrt{\frac{K}{11.676}} = 24.767$$

In this case, it turns out that the maximum applied duty cycle is approximately 50.26 %. So, the robot will likely not move very much. After some experimentation, the target settling time, T_s , was adjusted to 0.270 seconds to get the robot to apply a larger duty cycle on average. Some good values for K and a are selected as 11 601 and 29.630 respectively. In this scheme, the maximum duty cycle applied in the model is greater than 100 %. In reality, this is capped at 100 %. Furthermore, the compensation for the overshoot of the system is not sufficient to cause the robot to move backwards. The step response is shown in Figure 8 below. On closer examination, the duty cycle should drop below 50 % at approximately 0.8 seconds, at which point the robot is very near its target.

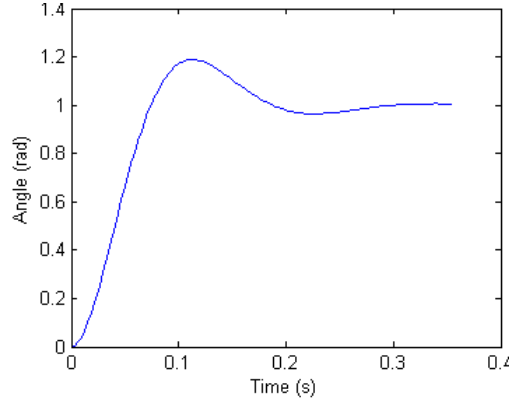


Figure 8: Step Response of Motor Controller

Note that significantly more experimentation needs to occur to tune the model to the real robot under actual operating conditions. This is nothing but a first approximation of the motor controller. Furthermore, as mentioned earlier, this control system only accounts for favourable conditions where movement of the wheels is predictable and unimpeded, such as a hardwood floor and unlike sand or a fluffy carpet. For these scenarios, a more sophisticated feedback system using the internal positioning system or rotation sensors is necessary. It is also worth mentioning that this same system can be applied to turning the robot. For example, when turning left, a positive reference angle can be sent to the right wheels and a negative reference angle can be sent to the left wheels.

Furthermore, a hard stop of the system occurs whenever the forward-facing infrared sensor detects an obstacle in the path. Since the controller is implemented in software, this just requires the current angle to be saved and the reference angle to be set to the current angle. More simply, the control system could just be stopped and the duty cycle set to zero, but this would not provide information on how far the robot has travelled based on current angle. Thus, the robot fulfills the functional specifications for movement.

3.6 - Robot Control Interface

The Robot Control Interface is intended to allow users to interact with the robot and to implement a distributed indoor positioning map such that multiple robots may share their knowledge of map topology. This will involve network communication, which is enabled by equipping robots with WiFi. The design should satisfy the following criteria:

1. A user is be able to command a robot to move to a position within the IPS coverage area from an arbitrary starting point to an ending point using a graphic user interface.
2. A user is able to graphically visualize the position of a robot within the range of the IPS as well as its travelled path and any obstacles that have been detected.
3. The robots are able to share information about their local maps in terms of obstacles found in order to build a distributed map accessible by all robots.

The following sections elaborate on how these criteria will be implemented. It is first necessary to choose a network layout for communication between the distributed system robots, servers and users. Based on the design of these sub-systems it is necessary to determine a form of data representation.

3.6.1 - Robot Network Topology and Synchronization

The organization of the robot network topology affects how the User Control Interface is implemented with respect to how and to whom the user will connect in order to control the robot. Since robots would, in effect, implement swarm intelligence by synchronizing their knowledge of map topology with each other, the map must be somehow stored, accessed and updated between these robots. Three solutions are evaluated using the following criteria:

1. Computation cost to update a robot's local map.
2. Computation cost to update the shared map.
3. Map consistency across robots.
4. Simplicity of communication.

Fully Distributed System

Each robot will be part of a distributed configuration. An example of a distributed system would have each robot correspond to a node within a structure similar to Chord Distributed Hash Table (DHT). Chord has nodes organized in a ring according to their identifier [10]. Each robot would be responsible for storing data regarding one area of the map, if a robot wants information regarding an area of the map that they themselves do not own, they would need to query the Chord DHT to access the robot that is responsible for that area. Should the robot find out that an obstacle exists in a location that was thought to be empty, it will send an update through the Chord DHT to the robot responsible for that area of the map. Thus the map is not stored in a centralized location and is stored across all robots in the system.

Through the Chord DHT, it is possible for users to send coordinates to robots in order to begin their path finding and movement algorithms towards that coordinate. However, this requires knowledge of one of the robot's MAC or IP address to communicate with to serve as an entry point into the distributed system (The request would be relayed to the correct robot through the distributed system). Problems occur since in certain networks, network interface cards may share the same public IP, which means that communication cannot be normally done with these cards. Using a MAC address to communicate with a robot is unsatisfactory since it is not necessarily known beforehand.

Broker System

Each robot may be located within a system in which they share a public IP address. To remedy this, we may need to send robot map sharing information to a server first, which can then relay this data to another robot. This system is similar to the Skype network protocol, where instead of two computers connecting directly to each other, they are able to connect via an intermediate relay server. More can be taken from the Skype protocol that would be useful here such as requiring authentication when first creating a connection with the server; this may stop hackers from inserting erroneous map information to the broker server. The broker server is defined as a server that is able to relay information and commands from one robot to another. Robots use it as an intermediate to communicate to another robot. In terms of robot path finding, the implementation would be as follows: Robots will be able to authenticate themselves with the broker server as well as choose to lookup other robots which have previously connected to the broker server.

In order for a user to communicate with a robot, a user will first contact the broker server, the broker server will check what robots are available and, if the requested entity is available, the broker server will be able to send a destination coordinates to the specified robot.

Client-Server Model

In a client-server model, the robots will communicate to a centralized server (web server). The server will be able to store representations of each robot as well as the most up to date map created from information across all robots. In order to share a map between robots, the process will work as follows: a robot will communicate with the centralized server. It will then compare the server's map with its own local map. Should the server contain cells with timestamps which are more up to date than its own, the robot will choose to update those cells within its local map. If the robot finds that its local cells are more up to date than the server's cells then it will update the cells on the server to its local values. A robot will update its own cells whenever it finds an obstacle. It is a simplistic design where the best map is stored in one location and robots are able to look at this shared location as well as update values for this shared location.

User Interface

Within each of these methods the user is able to communicate via a web application with the central server or distributed system. The web application will provide a graphical interface using JavaScript 2D graphics and will allow the user to enter coordinates to begin robot path finding and movement, the server (by having created a connection beforehand) will communicate with the specified robot to begin these algorithms.

Chosen System

Table 13 shows the criteria and evaluation of each system topology. Based on this analysis, it was found that the client-server model best fits the project requirements, and should be the most straightforward to implement. Thus, it was chosen as the final design.

Table 13: Summary of Topology Analysis

	Fully-Distributed	Broker	Client-Server
Cost to Update Local Map	If the area of the map to be updated is contained within this same robot than simply update it in constant time.	A robot is able to update its own version of the map in constant time.	The cost to update a robots local map is constant since it is stored within itself and no communication is needed.
Cost to Update Shared Map	Simply query for an update a specific position across the distributed system this is constant time.	Data will need to propagate between all robots. If a multicast tree is built then this takes linear time for amount of robots.	Simply update the shared map stored in the server, this will be constant time.
Map Consistency Across Robots	Will be consistent since there are no copies of the same map areas.	Will be copies of the same map data that may lead to inconsistencies in the map of different robots.	Transactions will only be made to the server so all views of the map will be identical since robots will contact the server for the map.
Simplicity of Communication	The user must already know at least one address which allows communication into the distributed system. However MAC addresses may not be known beforehand and a direct IP address may not be accessible due to robots having the same public IP addresses.	A user will ask the broker if a robot exist, if it does then a user can send command requests, this requires a connection be held between robot and server.	A user will only need to communicate with the central server to request a robot. Similarly to the broker the server must maintain connections or addresses of robots to relay commands.

3.6.2 - Data Representation

An efficient method must be used to respond to requests from users as well as service requests for information for robots within the system. The easiest means of implementing a query system, allowing concurrency and durability, is to use a relational database. The final design would include tables to represent robot ID's and data about the robot's position and path. The database would also information for the purpose of user authentication. The database contains a representation of the map and all of its individual cells along with their properties. This database will satisfy the requirements of the web application in which a user or a robot must be able to query robot or map information. The web application would be able to query this database in order to display these items within its graphical user interface.

The schema of the database was designed to store the discussed information. The entity relationship (ER) diagram in Figure 9 below was created to show the logical comparison between components. The schema was created thereof as can be seen below. The primary keys are underlined and foreign keys are italicized. Note that the diagram and schema are condensed for reasons of brevity. That is, other entities and relations exist in the full design.

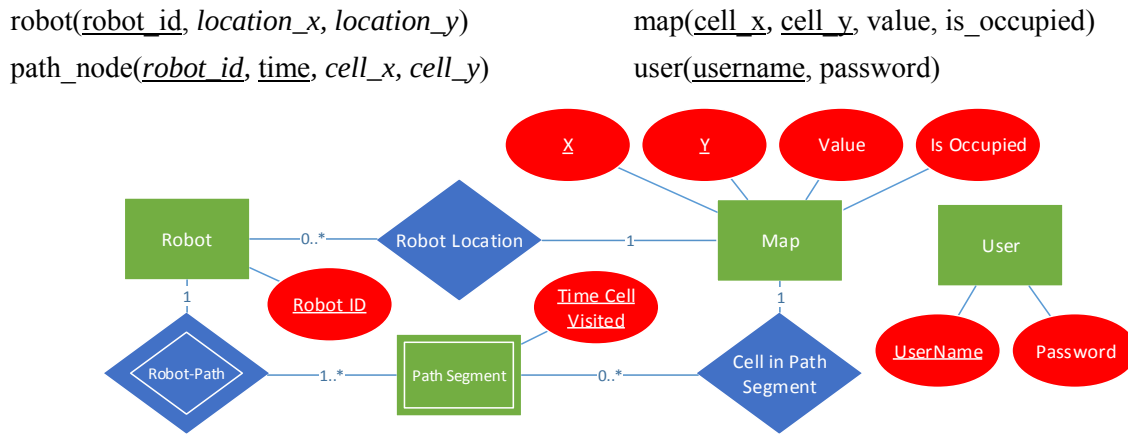


Figure 9: Entity Relationship Diagram for Robot Control Interface

In order to evaluate the schemas for data redundancies, it is permissible to check whether the database schema is in Boyce-Codd Normal Form (BCNF) or whether it is necessary to perform a decomposition into that form. Since BCNF is a lossless decomposition, it is ensured that data is not lost through such decomposition. Functional dependencies are used to evaluate BCNF given that the requirements are that $x \rightarrow y$ is a trivial relationship or x is a superkey [15]. Functional dependencies show that if two tuples within the database agree on the left hand side of $x \rightarrow y$ then both must agree on the right hand side. The functional dependencies for the schema are shown below. Since all left hand sides are super keys, this schema is in BCNF and therefore does not contain redundancy. Higher restrictions on redundancy such as 4NF are typically not viable or necessary for practical systems and are thus unnecessary.

$robot_id \rightarrow location_x, location_y$
 $robot_id, time \rightarrow cell_x, cell_y$
 $cell_x, cell_y \rightarrow value, is_occupied$
 $username \rightarrow password$

4 - Discussion on Design

4.1 - Discussion on Design

4.1.1 - Design Effort

At the time of the submission of this report all group members have contributed roughly equivalent effort on the reports and other deliverables, design and prototyping. The logs for each group member listed the following hours for each group member at the time of submission of this report:

(names removed)

4.1.2 - Analysis of Design

The design for this project spans many topics in Electrical and Computer Engineering. The Ultrasound Transceiver interface draws from ECE 240 and ECE 242, two courses that deal with electronic circuits. Furthermore, it uses knowledge from ECE 318, Analog and Digital Communications, and ECE 411, Digital Communications, to design an ultrasonic communication scheme. This section deals with many of the specifications that relate to the basic functionality of the indoor positioning system. It is meant to satisfy functional specification 3, beacon coverage, and functional specification 4, beacon detection range. This design element also contributes to functional specification 1, providing a coordinate system, and functional specifications 6 and 7, or accuracy of the IPS. Non-functional specifications 4 and 5, receiver and beacon size, are implicitly taken into account when designing the circuitry.

The design for ultrasound and RF media access control as well as beacon-receiver synchronization uses significant knowledge from ECE 454, Distributed Computing, in coordinating indoor position system operations. As a result, the system can operate properly without interference. Since much of this has to be implemented in embedded systems, it also draws on knowledge from ECE 455, Embedded Software, and ECE 224, Embedded Microprocessor Systems. Since significant thought is put into beacon and receiver networking, this section also uses knowledge from ECE 358, Computer Networks. This section primarily fulfills functional specification 1, IPS function, functional specification 2, multiple receivers, and functional specification 5, operation as a distributed system. Correct operation will also affect functional specifications 6 and 7, which deal with the accuracy of the indoor positioning system. The method used to synchronize ultrasonic communication fulfills functional specification 8, positional update frequency.

The speed of sound detector deals with the specific behaviour of mechanical waves under varying conditions. This uses knowledge that does not specifically come from a course in the program. However, some of the analysis was inspired by courses such as ECE 209, Electronic and Electrical Properties of Materials, in the sense that calculations for wavelength and travel time for sound had to take into account several physical conditions. The selection of components for embedded hardware also relates to ECE 224 and ECE 455. This section is important to functional specification 6 and 7, which deal with the accuracy of the IPS, since accuracy is largely dependent on a measurement of the travel time of sound.

The navigation and path-finding system deals with algorithmic design. In this case, ECE 457A, Cooperative and Adaptive Algorithms, is being used in two main ways. Firstly, a path to the destination must be calculated such that it avoids known obstacles as well as unexpected obstacles while actively looking for the shortest path. Second, the design accounts for the possibility of having multiple robots in operation. Here, the robots will cooperate to share a map of the area. Further algorithmic knowledge comes from ECE 250, Algorithms and Data Structures. This section is critical to fulfillment of functional specification 9, navigation.

The robot control system implements an analog controller. Thus, it draws knowledge from ECE 380, Analog Control Systems. Although it is not discussed in detail, much of the implementation of this system

occurs in embedded systems. So this section also uses knowledge from ECE 455 and ECE 224, both of which were mentioned earlier. This section is meant to fulfill functional specification 11, independent movement, and it implicitly fulfills specification 10, delivery payload. Along with the robot circuitry, power for the robot is briefly discussed. Thus, this section fulfills non-functional specification 2, robot battery life. The control system implemented here also takes into account unexpected obstacles, thereby fulfilling functional specification 12, stopping at obstacles.

Finally, the robot control interface deals with user input and a system to share information between multiple independent robots. This is implemented on a separate server and communication is established via a web application. As such, this section utilizes extensive knowledge from ECE 356, Database Systems, ECE 358, Computer Networks, and ECE 454, Distributed Computing. The primary goal of this section is to satisfy functional specifications 13 and 14, graphical user interface input and output.

The design for this project was done such that it should be able to meet all the functional requirements that were specified in this document. Many of these have already been discussed. However, a few requirements depend on the system as a whole and are therefore more difficult to pin down. These are functional specifications 6, 7, and 8, which deal with the accuracy and refresh rate of the indoor positioning system. Although they were mentioned above, these specifications require significant experimentation and fine-tuning in the actual implementation of the project.

It is difficult to predict if some of the non-functional specifications will be met based on theoretical design alone. Non-functional specifications 4, 5, 6 and 7 deal with the size and weight of the receivers and beacons. It is hard to say whether or not these will be met until the implementation is complete. Similarly, non-functional specification 3, peak current in the beacon, may differ between theory and practice.

One of the most creative parts of this design relates to the number of systems that are being used together to help make a solution more elegant than what can be seen today. Although indoor robots have become more common, most of them are unable to do the navigation in the way outlined in this report. Unlike most systems, this robot uses a beacon system and an A* algorithm to traverse a map. This addition of the indoor positioning system for navigation is a novel way to make the robot path planning more efficient as it allows the robot to know where it has been and whether it has gone off track. Including machine learning for obstacles, it presents an elegant and novel solution to a problem that is usually solved by random movement.

The design will most likely have to be refined later on since some experimental data is missing at this time. Development of the prototype is sure to reveal several unexpected problems. This detailed design document will be modified as problems are solved. A good example of a part of the project that will most likely need to be refined is the path planning algorithm. When designing the path planning algorithm, real world influences were taken into consideration. However, it is hard to judge how the mechanical components of the system will actually operate. During design, it was assumed that the other systems were working properly, which might not be the case. Therefore, depending on how the other systems work, the design for the path planning, among other systems, might have to change in order to deal with any inaccuracies.

4.1.3 - Potential Safety Hazards

Several possible safety concerns relate primarily to the use of custom circuitry, ultrasound, wireless and a moving robot. There is always intrinsic risk to electrical circuitry due to the potential for failure such as short circuiting which could result in damage or fire. Since the circuitry uses low voltages and low currents, the risk of direct harm such as electrical shock due to contact with circuitry is low. These hazards could be averted by using suitable fire and electricity resistant enclosures and fuses on power rails. It is worth noting that the ultrasound transceivers must be externally exposed, but direct contact should not be hazardous due to low voltages. Wireless communication may pose some risks due to signal interference.

because frequency bands such as 433 MHz, which are used by the prototype, are shared with other devices and may interfere with their operation.

Ultrasound poses a risk due to the high-intensity pulses that will be emitted at 40 kHz. 40 kHz is above human hearing ranges and thus not harmful. However, it does pose a possible risk of hearing damage to animals such as dogs, which may have hearing ranges extending above 40 kHz [16]. Because ultrasound attenuates very quickly and transmitters are directional, the risk should only exist in the vicinity that IPS is deployed. However, there is risk to people who rely on service animals if their animals react violently.

Finally, anything with moving parts such as a robot pose a risk to humans. In the case of the prototype, it is possible that the robot may collide with a human or fingers might become entrapped in a wheel when the robot is in motion. Both of these events could be mitigated by having the robot operate at a low speed and stopping the motors if stall currents are detected.

4.2 - Project Timeline

Table 14 provides a tentative schedule for the remainder of the project. The first thing that can be noted is that we will not finish receiving parts until July 14. This fact will not hurt the creation of the prototype since the project is easily broken up into different sections and some of these sections have all the necessary parts to create a prototype. As well some lower quality temporary parts have been acquired and these parts can be used in place of the ones that have not arrived yet. When the better parts arrive they can easily replace the lower quality parts. The goal for this term is to get 50% of the project complete by July 25 which will leave us time to finish the progress report. For our project, 50% completion is defined as getting the robot working such that it can detect objects with infrared, getting the IPS to calculate the location of the robot and getting the robot to move from one location to another using the calculated position.

After finishing the 50%, the parts that are remaining are the graphic user interface and the path planning algorithm that navigates around objects. Once the robot can be moved and steered, a navigation system is fairly simple and having a system to give a destination and get a location is simple in comparison to the other sections. Because of this, it is not included in the original 50%-75%. The aim is to have a working final prototype done by the end of the work term and then spend most of January testing and fixing any bugs encountered. After this, the rest of the time will be spent on the final report and preparing for the demo and symposium. Time has not been scheduled during the first half of August because of exam preparation. As well, the end of December is booked off in order to give time for work on work term reports and for the holidays. The plan is to start the final report during the work term to give plenty of time to get it done.

Table 14: Project Timeline

Description	Start	End	Duration
Obtain any remaining parts required for prototype	Jul. 01	Jul. 14	14 Days
Revise beacon design as prototyping finds issues	Jul. 04	Jul. 07	4 Days
Revise receiver design as prototyping finds issues	Jul. 04	Jul. 07	4 Days
Get basic robot control system working (i.e., robot can move and turn based on programed instructions)	Jul. 04	Jul. 12	9 Days
Robot is able to detect obstacles with infrared	Jul. 04	Jul. 12	9 Days
Finalize speed of sound detection approach	Jul. 04	Jul. 05	2 Days
Construct working Receiver prototype on breadboard	Jul. 07	Jul. 10	3 Days
Construct working Beacon prototype on breadboard	Jul. 07	Jul. 10	3 Days
Indoor Positioning System is functional (i.e, receiver can determine position in coordinate space)	Jul. 10	Jul. 12	2 Days
Refinement, testing and calibration of Indoor Positioning System to verify and meet specifications	Jul. 12	Jul. 18	2 Days
Robot can navigate to a destination using IPS with no obstacles	Jul. 08	Jul. 18	10 Days
Robot can detect and stop if obstacle encountered	Jul. 18	Jul. 20	2 Days

Progress Report	Jul. 20	Jul. 27	7 Days
Testing and debugging of system to be demoed	Jul. 20	Jul. 24	4 Days
Project Demo to Consultant	Jul. 25	Jul. 25	1 Days
Design and order printed circuit boards and components for Beacons, Receivers and Robot to be used for Design Symposium	Aug. 20	Sep. 30	40 Days
Choose enclosures for Beacons, Receivers and Robots	Aug. 20	Sep. 30	40 Days
Complete navigation of robot around obstacles	Sep. 01	Sep. 21	20 Days
Develop Robot Control Interface such that the robot can be issued tasks by a computer and its position is visible on screen	Sep. 01	Sep. 21	20 Days
Robot navigation with path learning is fully functional	Sep. 21	Oct. 07	17 Days
Assemble and Validate Beacon and Receiver Hardware	Sep. 30	Oct. 07	7 Days
Modify Robot Control Interface to allow map synchronization	Oct. 08	Oct. 15	7 Days
Testing and debugging, and verification of Specifications	Oct. 20	Nov. 15	26 Days
Preparation of Final Report	Nov. 15	Feb. 15	92 Days
Final testing and debugging for Final Demo and Symposium	Jan. 05	Feb. 01	26 Days
Preparation for Design Symposium	Feb. 01	Mar. 10	38 Days
Preparation for Final Demo	Feb. 01	Mar. 15	43 Days

References

- [1] International Federation of Robotics Statistical Department, "World Robotics 2013: Service Robots," International Federation of Robotics, Frankfurt, 2013.
- [2] J. Layton, "How Robotic Vacuums Work," howstuffworks, [Online]. Available: <http://electronics.howstuffworks.com/gadgets/home/robotic-vacuum2.htm>. [Accessed 2 June 2014].
- [3] United States of America Department of Defence, "Global Positioning System Standard Positioning Service Performance Standard," Department of Defence, Washington, DC, 2008.
- [4] A. Yazici, U. Yayan and H. Yucel, "An Ultrasonic Based Indoor Positioning System," in *Innovations in Intelligent Systems and Applications (INISTA)*, Istanbul, 2011.
- [5] N. B. Priyantha, "The Cricket Indoor Location System," Massachusetts Institute of Technology, Cambridge, 2005.
- [6] "PIC32MX250F128B," Microchip, 3 March 2014. [Online]. Available: <http://www.microchip.com/wwwproducts/Devices.aspx?dDocName=en557425>. [Accessed 3 July 2014].
- [7] "Damping of Air at High Frequencies," sengpielaudio.com, [Online]. Available: <http://www.sengpielaudio.com/calculator-air.htm>. [Accessed 3 July 2014].
- [8] "555 Timer IC," Wikipedia, 1 July 2014. [Online]. Available: http://en.wikipedia.org/wiki/555_timer_IC. [Accessed 3 July 2014].
- [9] "MAX3232," Maxim Integrated, January 2007. [Online]. Available: <http://www.maximintegrated.com/en/products/interface/transceivers/MAX3232.html>. [Accessed 3 July 2014].
- [10] A. S. Tanenbaum and M. V. Steen, *Distributed Systems: Principles and Paradigms* Second Edition, Upper Saddle River: Pearson Prentice Hall, 2007.
- [11] J. F. Kurose and K. W. Ross, *Computer Networking A Top-Down Approach*, New Jersey: Pearson, 2012.
- [12] "Speed of sound," Wikipedia, 29 June 2014. [Online]. Available: http://en.wikipedia.org/wiki/Speed_of_sound. [Accessed 1 July 2014].
- [13] O. Cramer, "The variation of the specific heat ratio and the speed of sound in air with temperature, pressure, humidity and CO2 Concentration," *Acoustic Society of America*, vol. 93, no. 5, pp. 2510-2516, 1993.
- [14] R. Jackett, "NPL Acoustics: Calculation of speed of sound in humid air," National Physical Laboratory, [Online]. Available: <http://resource.npl.co.uk/acoustics/techguides/speedair/>. [Accessed 1 July 2014].
- [15] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts* Sixth Edition, New York: McGraw-Hill, 2011.
- [16] "Hearing Range," Wikipedia, 25 June 2014. [Online]. Available: http://en.wikipedia.org/wiki/Hearing_range. [Accessed 7 July 2014].